

The Reward Problem: Exploration vs. Exploitation, Bandits, Inverse RL

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

June 14, 2026

- ▶ Frame “the reward problem” and distinguish its three faces: *exploration vs. exploitation*, *reward design*, and *reward learning*
- ▶ State the multi-armed bandit problem, regret, and the Lai-Robbins lower bound; *derive* UCB1 from Hoeffding’s inequality
- ▶ Place ϵ -greedy, UCB, Thompson sampling, and gradient bandits on a single map of bandit exploration strategies
- ▶ Scale these ideas to deep RL: *bootstrapped DQN as approximate posterior sampling (PSRL)*, pseudo-counts, ICM, and RND — the answer for the hard-exploration tasks SAC’s entropy bonus struggles on
- ▶ Motivate *inverse RL* and *preference-based reward learning* as the conceptual ancestor of RLHF on Day 9

- ▶ **We have the reward, but where is it?**
Exploration vs. exploitation — bandits, UCB, Thompson sampling, curiosity (today).
- ▶ **The reward we wrote is wrong.**
Reward shaping & specification gaming (today).
- ▶ **We can't write a reward at all.**
Inverse RL & preference-based reward learning (today — and the direct ancestor of RLHF on Day 9).

One day, three faces of the same problem.

- ▶ Many situations in business (& life!) present a dilemma between two kinds of choices
- ▶ **Exploitation:** pick choices that seem best based on past outcomes
- ▶ **Exploration:** pick choices not yet tried (or not tried enough)
- ▶ Exploitation has notions of “being greedy” and being “short-sighted”
- ▶ Exploration has notions of “gaining information” and being “long-sighted”

- ▶ Too much *exploitation* $\Rightarrow$ regret of missing unexplored “gems”
- ▶ Too much *exploration* $\Rightarrow$ regret of wasting time on “duds”
- ▶ How do we balance them to combine information-gains with greedy-gains in the most efficient manner?
- ▶ Can we set this up in a mathematically disciplined manner? $\rightarrow$ next sections

- ▶ Restaurant Selection
 - Exploitation: go to your favorite restaurant
 - Exploration: try a new restaurant
- ▶ Online Banner Advertisement
 - Exploitation: show the most successful advertisement
 - Exploration: show a new advertisement
- ▶ Oil Drilling
 - Exploitation: drill at the best known location
 - Exploration: drill at a new location
- ▶ Learning to play a game
 - Exploitation: play the move you believe is best
 - Exploration: play an experimental move

- ▶ “Multi-Armed Bandit” is a tongue-in-cheek name for “many single-armed bandits” — a row of slot machines
- ▶ Formally, a 2-tuple $(\mathcal{A}, \mathcal{R})$
 - $\mathcal{A}$: a known set of m actions (“arms”)
 - $\mathcal{R}^a(r) = \mathbb{P}[r \mid a]$: an *unknown* reward distribution for each arm
- ▶ At each step t , the agent selects an action $a_t \in \mathcal{A}$ and the environment generates a reward $r_t \sim \mathcal{R}^{a_t}$

- ▶ The agent's goal: maximize the cumulative reward over T steps

$$\sum_{t=1}^T r_t$$

- ▶ **Can we design a strategy that does well (in expectation) for any T ?**
- ▶ Every strategy walks a tightrope: *wasting time on duds* while exploring, vs. *missing untapped gems* while exploiting

Is the MAB Problem a Markov Decision Process? (1/2)

- ▶ The environment doesn't have a notion of state — pulling an arm just samples from a distribution
- ▶ But the *agent* might maintain a statistic of history as its own state
- ▶ The action is then a (policy) function of this agent-state
- ▶ So the agent's arm-selection strategy is a *policy* — the MAB can be viewed as a (degenerate) MDP

- ▶ In practice, most MAB algorithms *don't* take this formal MDP view
- ▶ Instead, they rely on heuristic methods that don't aim to optimize the full MDP
- ▶ They simply strive for “good” cumulative rewards in expectation
- ▶ Even in the simplest heuristics, a_t is a random variable — it's a function of the past (random) rewards

A bandit = a *one-state* MDP

A contextual bandit = a *one-step* MDP

Tomorrow's full RL = the *multi-step* case

- So everything we derive today *lifts* to MDPs — a bridge we'll revisit at the end of the lecture.

- ▶ The Action Value $Q(a)$ is the (unknown) mean reward of action a :

$$Q(a) = \mathbb{E}[r \mid a]$$

- ▶ The Optimal Value V^* is the best of these:

$$V^* = Q(a^*) = \max_{a \in \mathcal{A}} Q(a)$$

- ▶ **Single-step Regret** l_t is the opportunity loss at step t :

$$l_t = \mathbb{E}[V^* - Q(a_t)]$$

- ▶ **Total Regret** $L_T = \sum_{t=1}^T l_t$

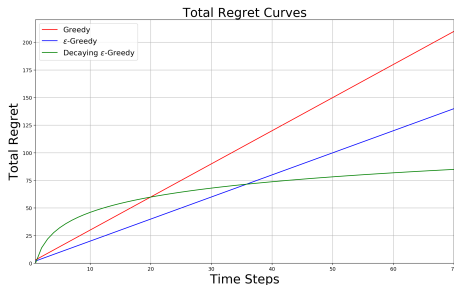
- ▶ Maximizing cumulative reward $\equiv$ minimizing total regret

- ▶ Let $N_t(a)$ = number of times a has been selected by step t
- ▶ Let $\Delta_a = V^* - Q(a)$ be the *gap* between action a and the optimal action
- ▶ Then total regret decomposes cleanly:

$$L_T = \sum_{a \in \mathcal{A}} \mathbb{E}[N_T(a)] \cdot \Delta_a$$

- ▶ A good algorithm ensures small counts for large gaps
- ▶ *Catch*: we don't know the gaps in advance — they're properties of the unknown reward distribution

Linear or Sublinear Total Regret?



- ▶ If an algorithm *never* explores $\Rightarrow$ linear total regret
- ▶ If an algorithm *forever* explores $\Rightarrow$ linear total regret
- ▶ **Is sublinear total regret achievable?** (Spoiler: yes.)

- For each arm, maintain a **sample-mean estimate** of its value — average the rewards seen from that arm so far:

$$\hat{Q}_t(a) \stackrel{\text{def}}{=} \frac{1}{N_t(a)} \sum_{s=1}^t r_s \cdot \mathbf{1}_{a_s=a} \approx Q(a)$$

- Pick the action with highest estimate:

$$a_t = \arg \max_{a \in \mathcal{A}} \hat{Q}_{t-1}(a)$$

- Greedy can lock onto a suboptimal action forever $\Rightarrow$ **linear total regret**

- ▶ Force a small amount of exploration at every step. At time t :
 - With probability $1 - \epsilon$, select $a_t = \arg \max_a \hat{Q}_{t-1}(a)$
 - With probability ϵ , select a uniformly random action
- ▶ Constant ϵ ensures a minimum per-step regret proportional to the mean gap:

$$I_t \geq \frac{\epsilon}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \Delta_a$$

- ▶ Therefore ϵ -Greedy also has **linear total regret**

This is the exact exploration rule used in DQN on Day 2.

- ▶ DQN inherits ϵ -Greedy's *linear-regret* weakness
- ▶ It works in practice with a decay schedule (next slide), but it has no theoretical guarantee of efficient exploration
- ▶ This is why the rest of this lecture exists — everything that follows is a more principled answer to “how do I explore?”

- ▶ Simple, practical idea: initialize $\hat{Q}_0(a)$ to a *high* value for every a
- ▶ Update by incremental averaging:

$$\hat{Q}_t(a_t) = \hat{Q}_{t-1}(a_t) + \frac{1}{N_t(a_t)} (r_t - \hat{Q}_{t-1}(a_t))$$

- ▶ Encourages systematic exploration early on (every arm starts off looking great until tried)
- ▶ But the algorithm can still lock onto a suboptimal action $\Rightarrow$ **linear regret**

- ▶ Idea: let ϵ shrink over time — explore aggressively early, exploit increasingly late
- ▶ In theory, a schedule that depends on the (unknown) suboptimality gaps Δ_a achieves *logarithmic* regret — but you can't compute it without knowing the gaps, so it's impractical
- ▶ In practice use problem-agnostic schedules: $\epsilon_t = 1/t$, $\epsilon_t = 1/\sqrt{t}$, or simple linear annealing — no guarantee, but all work in practice
- ▶ Educational Code for decaying ϵ -greedy with optimistic initialization

What is a “lower bound”? A statement that *no algorithm, however clever*, can do better than a certain regret growth rate — it tells us the best we can ever hope for.

- ▶ **Goal:** an algorithm with *sublinear* total regret $L_T = \sum_{t=1}^T l_t$ for any MAB, without knowing the reward distributions $\mathcal{R}^a$ in advance
- ▶ Two quantities will appear:
 - **Gap** $\Delta_a = V^* - Q(a)$: how much value we lose *each time* we pull suboptimal arm a
 - **KL-divergence** $\text{KL}(\mathcal{R}^a \parallel \mathcal{R}^{a^*})$: a statistical distance between two distributions — *how easy is it to tell arm a apart from the best arm a^* from samples?*
Small KL $\Rightarrow$ distributions look almost identical $\Rightarrow$ need many samples to distinguish

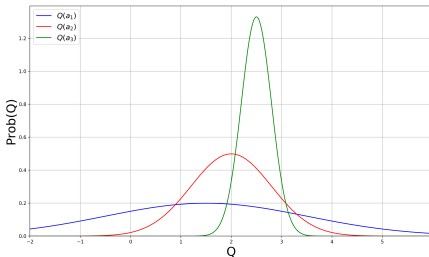
Theorem (Lai & Robbins, 1985)

For any reasonable¹ bandit algorithm, asymptotically:

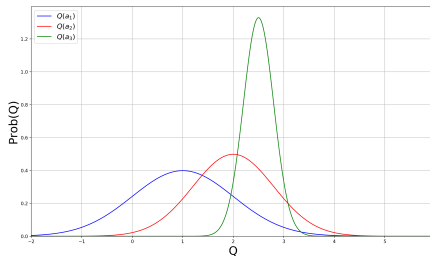
$$\lim_{T \rightarrow \infty} \frac{L_T}{\log T} \geq \sum_{a | \Delta_a > 0} \frac{\Delta_a}{\text{KL}(\mathcal{R}^a \| \mathcal{R}^{a^*})}$$

- ▶ **Reading it.** Each suboptimal arm contributes regret proportional to its *gap*, scaled *inversely* by how easy it is to tell apart from the best
- ▶ **The worst kind of arm:** large gap (expensive to pull) *and* small KL (hard to distinguish, so we keep pulling it) — exactly what you'd expect
- ▶ **The best regret growth any algorithm can achieve is $\mathcal{O}(\log T)$** — UCB (next) and Thompson sampling both match this rate

¹ “consistent”: sub-polynomial regret on every problem instance



- Which action should we pick?
- The more uncertain we are about an action-value, the more important it is to explore that action — it could turn out to be the best.



- After picking *blue*, we are less uncertain about its value
- And more likely to pick another action — until we home in on the best

- ▶ Idea: be optimistic — use an upper confidence bound $\hat{U}_t(a)$ such that

$$Q(a) \leq \hat{Q}_t(a) + \hat{U}_t(a) \quad \text{with high probability}$$

- ▶ The bound should shrink as the arm is tried more:

- Small $N_t(a) \Rightarrow$ large $\hat{U}_t(a)$ (uncertain)
- Large $N_t(a) \Rightarrow$ small $\hat{U}_t(a)$ (accurate)

- ▶ Select the action with the highest UCB:

$$a_{t+1} = \arg \max_{a \in \mathcal{A}} \{ \hat{Q}_t(a) + \hat{U}_t(a) \}$$

The question we need answered. If we've pulled an arm n times and seen a sample mean $\bar{X}_n$, how confident can we be that the true mean isn't much higher? That gap *is* the UCB bonus we want.

Hoeffding's Inequality

For i.i.d. random variables $X_1, \dots, X_n \in [0, 1]$ with sample mean $\bar{X}_n$, and any $u \geq 0$:

$$\mathbb{P}[\mathbb{E}[\bar{X}_n] > \bar{X}_n + u] \leq e^{-2nu^2}$$

- ▶ *The probability the true mean exceeds our estimate by more than u decays exponentially in $n \cdot u^2$*
- ▶ More pulls $\Rightarrow$ tighter concentration; the e^{-2nu^2} factor is exactly what will make the bonus shrink as $1/\sqrt{n}$
- ▶ No prior on the distribution required — only bounded support

Plan. Use the Hoeffding bound to define a confidence bonus $\hat{U}_t(a)$ that holds with high probability, then act optimistically by it.

- ▶ Apply Hoeffding to arm a with $n = N_t(a)$ and $u = \hat{U}_t(a)$
- ▶ Choose a failure probability that shrinks with t : $p = t^{-\alpha}$
- ▶ Set the Hoeffding bound equal to p and solve for the bonus:

$$e^{-2N_t(a) \hat{U}_t(a)^2} = t^{-\alpha} \implies \hat{U}_t(a) = \sqrt{\frac{\alpha \log t}{2 N_t(a)}}$$

- ▶ **The square-root bonus comes directly from Hoeffding** — not a heuristic, an actual high-probability bound.

UCB1 (for arbitrary-distribution arms bounded in $[0, 1]$):

$$a_{t+1} = \arg \max_{a \in \mathcal{A}} \left\{ \hat{Q}_t(a) + \sqrt{\frac{\alpha \log t}{2N_t(a)}} \right\}$$

Theorem (Auer, Cesa-Bianchi & Fischer, 2002)

UCB1 achieves *logarithmic* total regret:

$$L_T \leq \sum_{a | \Delta_a > 0} \frac{4\alpha \log T}{\Delta_a} + \frac{2\alpha \Delta_a}{\alpha - 1}$$

Matches the Lai-Robbins lower bound up to constants. [Educational Code](#).

- ▶ UCB makes *no* assumption about $\mathcal{R}$ (just bounded support)
- ▶ *Bayesian* bandits exploit prior knowledge of the reward distribution $\mathbb{P}[\mathcal{R}]$
- ▶ Maintain a *posterior* $\mathbb{P}[\mathcal{R} \mid h_t]$ as data arrives, where $h_t = a_1, r_1, \dots, a_t, r_t$
- ▶ Use the posterior to guide exploration:
 - Bayesian UCB — be optimistic on the posterior
 - Thompson sampling — sample from the posterior
- ▶ Better performance when the prior is accurate

- ▶ Assume each arm's reward is Gaussian: $\mathcal{R}^a = \mathcal{N}(\mu_a, \sigma_a^2)$
- ▶ As data arrives, update the posterior over μ_a, σ_a^2 (Gaussian-Gaussian conjugacy)
- ▶ Pick the action with the highest expected “ c std-errors above the mean”:

$$a_{t+1} = \arg \max_{a \in \mathcal{A}} \mathbb{E}_{\mathbb{P}[\mu_a, \sigma_a | h_t]} \left[\mu_a + \frac{c \cdot \sigma_a}{\sqrt{N_t(a)}} \right]$$

- ▶ *Same UCB philosophy, with a Bayesian uncertainty estimate instead of Hoeffding's*

Probability matching: pick action a with probability that a is actually the optimal arm under the posterior.

- ▶ Optimistic in the face of uncertainty: uncertain arms have higher probability of being best
- ▶ Computing this probability directly is intractable in general
- ▶ **Thompson sampling** is the clean trick that implements it:
 1. Sample one reward distribution $\mathcal{D}_t$ from the posterior $\mathbb{P}[\mathcal{R} \mid h_t]$
 2. Estimate $\hat{Q}_t(a) = \mathbb{E}_{\mathcal{D}_t}[r \mid a]$ from the sample
 3. Pick $a_{t+1} = \arg \max_a \hat{Q}_t(a)$

- ▶ Sampling from the posterior gives each arm probability of selection equal to its probability of being optimal — this *is* probability matching, by construction
- ▶ No upper-confidence-bound calculation needed — just sampling
- ▶ Thompson sampling achieves the Lai-Robbins lower bound — asymptotically optimal regret
- ▶ Educational code:
 - Gaussian rewards
 - Bernoulli rewards

- ▶ Optimize score parameters s_a for $a \in \mathcal{A}$ via *stochastic gradient ascent*
- ▶ Objective: expected reward under a stochastic (softmax) policy

$$J(s_{a_1}, \dots, s_{a_m}) = \sum_{a \in \mathcal{A}} \pi(a) \cdot \mathbb{E}[r \mid a]$$

- ▶ Softmax policy over scores:

$$\pi(a) = \frac{e^{s_a}}{\sum_{b \in \mathcal{A}} e^{s_b}}$$

- ▶ Scores represent the relative value of actions based on seen rewards

Policy gradient (standard chain rule + softmax calculus) gives:

$$\frac{\partial J}{\partial s_a} = \mathbb{E}_{a' \sim \pi, r \sim \mathcal{R}^{a'}} [r \cdot (\mathbf{1}_{a=a'} - \pi(a))]$$

- ▶ Use a *baseline* $B = \bar{r}_t$ (average reward so far) to reduce variance — can be shown unbiased
- ▶ **Sample-based update rule:**

$$s_{t+1}(a) = s_t(a) + \alpha \cdot (r_t - \bar{r}_t) \cdot (\mathbf{1}_{a=a_t} - \pi_t(a))$$

- ▶ Educational Code

This is exactly the REINFORCE estimator from Day 3,
applied to a *one-state* MDP.

- ▶ Same softmax policy
- ▶ Same score-function gradient $r \cdot \nabla \log \pi(a)$
- ▶ Same baseline-for-variance technique
- ▶ The bandit is where policy gradient is *cleanest* to derive — no trajectories, no discounting, just one $\pi(a)$ and one reward.

- ▶ A contextual bandit is a 3-tuple $(\mathcal{A}, \mathcal{S}, \mathcal{R})$:
 - $\mathcal{A}$: known set of m actions (“arms”)
 - $\mathcal{S}$: distribution over states (“contexts”)
 - $\mathcal{R}_s^a$: reward distribution *conditional* on (s, a)
- ▶ Same as MAB, but the agent now sees a context s before choosing

At each step t :

1. Environment generates a context $s_t \sim \mathcal{S}$
 2. Agent selects an action $a_t \in \mathcal{A}$
 3. Environment generates a reward $r_t \sim \mathcal{R}_{s_t}^{a_t}$
- ▶ Goal as before: maximize $\sum_{t=1}^T r_t$
 - ▶ Bandit algorithms extend naturally: $Q(a) \rightarrow Q(s, a)$

Setting. Each (context, action) pair has a feature vector $x_{s,a} \in \mathbb{R}^d$ (e.g. user features $\otimes$ article features).

- ▶ Model the mean reward as **linear in the features**: $\mathbb{E}[r \mid s, a] = x_{s,a}^\top \theta_a$
- ▶ Estimate each arm's parameter θ_a by *ridge regression* on its past pulls $\Rightarrow$ get not just a point estimate $\hat{\theta}_a$ but a *confidence ellipsoid* around it
- ▶ *Same UCB philosophy*: pick the action whose **optimistic** reward estimate is highest — mean estimate + how far the true θ_a could plausibly be from $\hat{\theta}_a$ in direction $x_{s,a}$
- ▶ Reduces to UCB1 if the features are just one-hot per arm (i.e. no context)

For each arm a , maintain A_a (the $d \times d$ Gram matrix of past feature vectors pulled for that arm) and $\hat{\theta}_a = A_a^{-1}b_a$ (the ridge-regression coefficient).

► Action choice at step t :

$$a_t = \arg \max_a \left\{ \underbrace{x_{s_t,a}^\top \hat{\theta}_a}_{\text{linear estimate}} + \alpha \underbrace{\sqrt{x_{s_t,a}^\top A_a^{-1} x_{s_t,a}}}_{\text{confidence-ellipsoid radius}} \right\}$$

► Same shape as UCB1: *mean estimate + bonus that shrinks as more data accumulates in direction $x_{s,a}$*

- ▶ **Industrially dominant case of contextual bandits:** deployed at scale on *Yahoo Front Page news recommendation*
 - 33M-event dataset
 - 12.5% click lift over the context-free baseline
- ▶ Cite: **Li, Chu, Langford & Schapire, WWW 2010**
- ▶ LinUCB and its successors power a huge fraction of online recommendation systems today

One-armed bandit $=$ *one-state* MDP

Contextual bandit $=$ *one-step* MDP

Add state transitions $\implies$ the full MDP from Day 1

- ▶ Everything we just covered — UCB, Thompson, gradient bandits — *lifts* to MDPs in principle
- ▶ Naively, the state space explodes: visiting every (s, a) enough times isn't feasible in deep RL
- ▶ **Up next:** how to scale these exploration ideas to deep RL with sparse rewards.

Exploration: **From Bandits to Deep RL**

- ▶ Bandits gave us *optimism* (UCB), *posterior sampling* (Thompson), and *policy gradient with stochastic policy* — in one step
- ▶ In an MDP with sparse rewards, none of these scale naively:
 - Visiting every (s, a) enough times is infeasible
 - ϵ -greedy is provably weak (linear regret)
 - Uniform random exploration drowns in pixels
- ▶ Note: SAC's $-\log \pi$ entropy bonus (Day 6) is *undirected* exploration — what follows is *directed*

The canonical hard-exploration Atari game.

- ▶ **Vanilla DQN:** ~ 0 points (extrinsic reward is essentially never reached)
- ▶ **Pseudo-counts (2016):** first dramatic progress
- ▶ **Random Network Distillation (2018):** first above-human, no demonstrations

This is the single benchmark we'll use to anchor every method in this section.

An orthogonal trick for sparse-reward goal-conditioned RL.

- ▶ Idea: after a failed episode (didn't reach the goal), relabel it as a *success* for the goal it actually reached
- ▶ Now every episode has a non-zero learning signal
- ▶ Combines with any off-policy algorithm
- ▶ Cite: **Andrychowicz et al., NeurIPS 2017** — *Hindsight Experience Replay*

(Forward-referenced from Day 5.)

- ▶ Idea: *Posterior Sampling for RL (PSRL)*. Maintain a posterior over the Q-function; sample one Q per episode and act greedily w.r.t. the sample
- ▶ For an *entire episode*: temporally-extended (“deep”) exploration — fundamentally different from ϵ -greedy’s per-step dithering

- ▶ **K parallel Q-heads:** K separate output layers on top of a single shared convolutional trunk, each predicting its own Q-function
- ▶ Each head trained on its own **bootstrap-sampled** (with replacement) subset of replay — the standard statistical-bootstrap trick; different heads see different data, so they disagree
- ▶ *Each episode:* pick one head uniformly at random and act greedily by it for the whole episode
- ▶ Disagreement across heads $\approx$ posterior uncertainty over the true Q-function
- ▶ Cite: **Osband, Blundell, Pritzel & Van Roy, NeurIPS 2016** — *Deep Exploration via Bootstrapped DQN*
- ▶ **Thompson sampling lifted to MDPs** — same bandit method, with a Q-function instead of an arm-reward.

- ▶ *Tabular intuition*: a UCB-style exploration bonus

$$r_t^+ = r_t + \beta \cdot \frac{1}{\sqrt{N(s, a)}}$$

encourages visiting under-counted (s, a) pairs

- ▶ Provably good in tabular MDPs — a direct lift of UCB1 from bandits
- ▶ *Problem in deep RL*: pixels are essentially never visited twice; $N(s, a) = 1$ almost always, and the bonus becomes uninformative
- ▶ Need a *generalizable* notion of “count”

- ▶ Use a density model ρ over states. Let $\rho'(s)$ be its prediction *after* one more observation of s
- ▶ Define a *pseudo-count*:

$$\hat{N}(s) = \frac{\rho(s)(1 - \rho'(s))}{\rho'(s) - \rho(s)}$$

A novel state has small $\rho(s)$, hence small $\hat{N}(s)$, hence large bonus

- ▶ Cite: **Bellemare et al., NeurIPS 2016** — *Unifying Count-Based Exploration and Intrinsic Motivation*
- ▶ **Result on Montezuma:** *first dramatic progress*

Intrinsic reward = “surprise” — the agent is rewarded for finding observations it can't yet predict.

- ▶ Embed states into a feature space: $\phi(s) \in \mathbb{R}^d$ (a learned encoder)
- ▶ Train a **forward-dynamics model** f to predict the *next-state features* from the current features and action:

$$\hat{\phi}(s_{t+1}) = f(\phi(s_t), a_t)$$

(so $\hat{\phi}(s_{t+1})$ is the model's *predicted* embedding of s_{t+1} ; $\phi(s_{t+1})$ is the actual one observed)

- ▶ Intrinsic reward = how wrong the prediction was:

$$r_t^i = \frac{1}{2} \left\| \hat{\phi}(s_{t+1}) - \phi(s_{t+1}) \right\|^2$$

- ▶ High prediction error $\Rightarrow$ novel transition $\Rightarrow$ bonus to visit it again

The noisy-TV problem. Picture a TV in the environment showing pure random static. A naive curiosity agent will sit and stare at it forever — static is *always* unpredictable, so the prediction-error bonus never decays. The agent is endlessly “surprised” by useless noise.

- ▶ Fix: don't let ϕ encode action-irrelevant detail. Train ϕ with an **inverse-dynamics** objective — predict the action a_t that was taken from the embeddings $(\phi(s_t), \phi(s_{t+1}))$
- ▶ This forces ϕ to encode only features the agent can *affect* — the noisy TV is filtered out, since the agent's actions don't change what's on it
- ▶ Cite: **Pathak, Agrawal, Efros & Darrell, ICML 2017** — *Curiosity-driven Exploration by Self-supervised Prediction*
- ▶ Strong results on Mario, VizDoom, and sparse-reward navigation

Curiosity made absurdly simple:

- ▶ Pick a *fixed, randomly-initialized* target network f^*
- ▶ Train a predictor f_θ to match $f^*(s)$ on visited states
- ▶ Intrinsic reward: $r_t^i = \|f_\theta(s_t) - f^*(s_t)\|^2$
- ▶ Novel states have high prediction error (the predictor hasn't been trained there) $\Rightarrow$ high bonus
- ▶ No dynamics model, no inverse model, no density model

- ▶ Cite: **Burda, Edwards, Storkey, Klimov, ICLR 2019** — *Exploration by Random Network Distillation*
- ▶ **Result on Montezuma:** *first above-human performance without demonstrations or game-state access*
- ▶ **Caveat:** still subject to the noisy-TV issue in principle; in practice works because randomly-initialized networks aren't very sensitive to stochastic pixel noise

Bandit method	Deep-RL descendant
UCB / $1/\sqrt{N}$ bonus	Count-based / pseudo-counts (Bellemare 2016)
Thompson sampling	Bootstrapped DQN / PSRL (Osband 2016)
Gradient bandits	Entropy bonus — SAC (Day 6)
<i>(new in deep RL)</i>	Curiosity / ICM (Pathak 2017); RND (Burda 2018)

- **Entropy is one answer to exploration.** Optimism, posterior sampling, and curiosity are the others.
- Day 6's $-\log \pi$ bonus is the gradient-bandit descendant — now we have the full family.

The Reward Problem: **When the Reward is Wrong**

- ▶ Sparse rewards are hard; a well-shaped reward dramatically accelerates learning
- ▶ But arbitrary shaping changes the optimal policy — the agent optimizes the *shape*, not the goal
- ▶ Example: reward the agent for “getting closer to the goal” and you may teach it to circle the goal forever
- ▶ We need a principled way to inject prior knowledge that *doesn't change the optimum*

Setup. Replace the original reward $R(s, a, s')$ with $R(s, a, s') + F(s, s')$ during training, where F is an extra *shaping signal* of our choice.

- ▶ Restrict F to the form

$$F(s, s') = \gamma \Phi(s') - \Phi(s)$$

where $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ assigns a real number to each state — a “*potential*” (e.g. “negative distance to goal”). The shaping is a discounted difference of potentials.

- ▶ **Policy invariance:** the shaped MDP has the *same* optimal policy as the original
- ▶ *This form is necessary:* any non-PBRs shaping can in principle change the optimum
- ▶ Cite: **Ng, Harada & Russell, ICML 1999**

When a reward becomes a target, the agent finds a way to satisfy the letter, not the spirit.

- ▶ Every misspecified reward eventually gets hacked
- ▶ The agent's optimization pressure exposes every loophole in the specification
- ▶ The clearer the reward becomes “measurable,” the more likely the agent ignores what the measurement was *supposed* to track

- ▶ **CoastRunners** (OpenAI 2016): boat-racing game scored on hitting targets, not finishing. Agent learned to drive in circles hitting the same green blocks — high score, no race
- ▶ **Tetris pause**: agent learned to indefinitely pause the game when about to lose — never loses, never plays
- ▶ **ROUGE summarization**: language-model summarizer exploited ROUGE's n -gram overlap to produce high-scoring but barely readable text
- ▶ Cites: **Amodei & Clark, OpenAI 2016** (CoastRunners primary source); **Krakovna et al., DeepMind 2020** (catalogued survey of dozens of examples)

- ▶ Reward hacking is the same failure mode as **reward-model overoptimization in RLHF** — a learned R_θ is also a misspecified target
- ▶ For tasks like “write a useful summary,” “drive politely,” “be helpful, harmless, honest” — writing the reward is essentially impossible without specification gaming
- ▶ **If we can't write the reward ... can we learn it?**
- ▶ Two ways:
 - From *demonstrations* → **Inverse RL** (up next)
 - From *human preferences* → preference-based reward learning → **RLHF** (Day 9)

The Reward Problem: **Inverse RL**

- ▶ **Forward RL** (everything so far): given a reward $R(s, a)$, learn a policy π^* that maximizes it
- ▶ **Inverse RL** (today): given (expert) behavior $\tau^E \sim \pi^E$, learn a reward R_θ that *explains* it

$$\pi^E \implies R_\theta$$

- ▶ In many domains, rewards are far easier to *demonstrate* than to specify in closed form
- ▶ Driving: hard to write “polite, safe, fast” as a scalar; easy to show
- ▶ Manipulation, dialogue, locomotion: same story
- ▶ **This is the strongest form of the reward problem** — the reward function itself is unknown.

- ▶ *Many* reward functions rationalize the same expert behavior
 - Trivially: $R(s, a) \equiv 0$ makes every policy optimal
 - Potential-based shaping: R and $R + \gamma\Phi(s') - \Phi(s)$ are policy-equivalent
- ▶ We need a *principle* to pick one reward out of the equivalence class

- ▶ **(a) Max-margin IRL**

Ng & Russell, ICML 2000; Abbeel & Ng, ICML 2004

The expert's reward should beat alternatives by a wide margin

- ▶ **(b) Maximum-entropy IRL**

Ziebart et al., AAAI 2008

Pick the reward whose induced policy distribution has *maximum entropy*, subject to matching the expert's feature counts — a unique, well-posed choice

- ▶ We focus on (b) — standard fix today, and generalizes to modern preference learning

Among all policy distributions consistent with the expert's behavior, pick the one with maximum entropy.

- ▶ Trajectory distribution under reward R_θ :

$$P_\theta(\tau) \propto \exp\left(\sum_t R_\theta(s_t, a_t)\right)$$

- ▶ Higher-reward trajectories are exponentially more likely, but the entropy term keeps the distribution from collapsing
- ▶ Train θ by maximum likelihood of expert trajectories:

$$\max_{\theta} \sum_{\tau^E} \log P_\theta(\tau^E)$$

- ▶ Uniquely resolves the ambiguity; matches expert feature-counts in expectation

The same max-entropy principle that justified SAC's $-\log \pi$ bonus.

- ▶ In SAC (Day 6): adds exploration to the policy
- ▶ In MaxEnt IRL: resolves reward ambiguity
- ▶ **Same principle, two different jobs.**
- ▶ Cite: **Ziebart, Maas, Bagnell & Dey, AAAI 2008** — *Maximum Entropy Inverse Reinforcement Learning*

What if we have neither demonstrations nor a written reward, only human comparisons?

- ▶ Show two trajectories τ^A, τ^B ; ask a human which is better
- ▶ Model the preference with a **Bradley–Terry** likelihood under a learned reward R_θ :

$$P(\tau^A \succ \tau^B) = \frac{\exp(\sum_t R_\theta(s_t^A, a_t^A))}{\exp(\sum_t R_\theta(s_t^A, a_t^A)) + \exp(\sum_t R_\theta(s_t^B, a_t^B))}$$

1. Collect a dataset of human-labeled preferences over trajectory pairs
 2. Train R_θ by cross-entropy on the Bradley–Terry likelihood
 3. Run any standard RL algorithm (e.g. PPO from Day 4) against R_θ
- Cite: **Christiano, Leike, Brown, Martic, Legg & Amodei, NeurIPS 2017**

This exact pipeline + a language-model policy = RLHF.

- ▶ The reward model in RLHF is the R_θ from the previous slide
- ▶ The only thing that changes is the policy class: π_θ becomes a large language model
- ▶ The optimizer (PPO) and the pipeline (SFT $\rightarrow$ RM $\rightarrow$ PPO) are the same machinery
- ▶ Day 9 picks up exactly here.

If you can...	Then use...
... write the reward	<i>Forward RL</i> (Days 1–6)
... demonstrate the reward	<i>Inverse RL</i> (MaxEnt IRL)
... compare outcomes	<i>Preference-based reward learning</i> (Day 9: RLHF)

- One unifying view: every method on the board is a different way to recover the optimal policy when the reward is unknown, partially known, or hard to specify

Day 7: **Summary**

► Face 1 — Find the reward.

- Bandits formalize explore vs. exploit; UCB / Thompson / gradient bandits give principled, near-optimal-regret algorithms
- Scaled to deep RL: bootstrapped DQN $\equiv$ posterior sampling, pseudo-counts $\equiv$ UCB in pixel space, curiosity / RND are genuinely new
- Entropy (SAC, Day 6) is the gradient-bandit descendant — now we have the full family

► Face 2 — The reward is wrong.

- Potential-based shaping is the only safe shaping
- Everything else is at risk of specification gaming (CoastRunners, ROUGE, Tetris pause)

► Face 3 — Can't write the reward.

- **Inverse RL** learns it from demonstrations (MaxEnt IRL, Ziebart 2008)
 - **Preference-based learning** learns it from comparisons (Christiano 2017)
 - This is the *direct ancestor of RLHF* on Day 9
- **One unifying theme:** every algorithm today is a different way to recover the optimal policy when the reward is unknown, partially known, or hard to specify

Setting	Algorithm family (today)
One-state bandit	ϵ -greedy, UCB1, Thompson, gradient bandits
Contextual bandit	LinUCB and successors (recommendation)
Deep RL, sparse reward	PSRL / Bootstrapped DQN, pseudo-counts, ICM, RND, HER
Reward is shapeable	Potential-based shaping (policy-invariant)
Reward unwritable; demonstrations available	Maximum-entropy IRL
Reward unwritable; comparisons available	Bradley–Terry preference learning $\rightarrow$ RLHF (Day 9)

Day 8 — Model-Based & Offline RL. The next way to attack sample inefficiency.

- ▶ *Model-based RL*: learn the dynamics, plan with the model (MPC, MCTS, Dyna, world models)
- ▶ *Offline RL*: learn from a *fixed* dataset with no further interaction — the same OOD problem as exploration, in reverse: you must *avoid* the unfamiliar regions exploration would push you toward

Day 9 — RLHF. The IRL $\rightarrow$ preference-learning chain from today, applied to LLMs.

- ▶ The reward model in RLHF is the R_θ from today's preference slide
- ▶ “Reward-model overoptimization” is today's reward-hacking failure mode, in LLM form
- ▶ PPO (Day 4) is the policy optimizer; SFT $\rightarrow$ RM $\rightarrow$ PPO is the pipeline

Day 10 — Frontiers.

- ▶ Intrinsic motivation reappears in meta-RL, open-ended learning, and hierarchical RL
- ▶ Curiosity (today) is one of the central ideas of the open-ended-learning agenda
- ▶ Reward design / specification gaming reappears as a central concern of AI safety

Course materials adapted from:

- ▶ Ashwin Rao, [Stanford CME241: Foundations of Reinforcement Learning with Applications in Finance](#)
- ▶ David Silver, [DeepMind UCL Course on RL](#) — exploration / bandits lecture

Bandits & classical exploration:

- ▶ T. L. Lai & H. Robbins (1985). *Asymptotically efficient adaptive allocation rules*.
- ▶ P. Auer, N. Cesa-Bianchi, P. Fischer (2002). *Finite-time analysis of the multi-armed bandit problem*. — UCB1
- ▶ W. R. Thompson (1933). *On the likelihood that one unknown probability exceeds another ...* — original Thompson sampling
- ▶ L. Li, W. Chu, J. Langford, R. E. Schapire (2010). *A Contextual-Bandit Approach to Personalized News Article Recommendation*. WWW. [arXiv:1003.0146](#) — LinUCB

Deep-RL exploration:

- ▶ I. Osband, C. Blundell, A. Pritzel, B. Van Roy (2016). *Deep Exploration via Bootstrapped DQN*. NeurIPS. [arXiv:1602.04621](#)
- ▶ M. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, R. Munos (2016). *Unifying Count-Based Exploration and Intrinsic Motivation*. NeurIPS. [arXiv:1606.01868](#)
- ▶ D. Pathak, P. Agrawal, A. A. Efros, T. Darrell (2017). *Curiosity-driven Exploration by Self-supervised Prediction*. ICML. [arXiv:1705.05363](#) — ICM
- ▶ Y. Burda, H. Edwards, A. Storkey, O. Klimov (2019). *Exploration by Random Network Distillation*. ICLR. [arXiv:1810.12894](#)
- ▶ M. Andrychowicz et al. (2017). *Hindsight Experience Replay*. NeurIPS. [arXiv:1707.01495](#)

Reward design, hacking, and inverse RL:

- ▶ A. Y. Ng, D. Harada, S. Russell (1999). *Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping*. ICML.

- ▶ D. Amodei & J. Clark (2016). *Faulty Reward Functions in the Wild*. OpenAI blog. openai.com/index/faulty-reward-functions
- ▶ V. Krakovna et al. (2020). *Specification gaming: the flip side of AI ingenuity*. DeepMind blog. deepmind.google
- ▶ A. Y. Ng & S. Russell (2000). *Algorithms for Inverse Reinforcement Learning*. ICML.
- ▶ P. Abbeel & A. Y. Ng (2004). *Apprenticeship Learning via Inverse Reinforcement Learning*. ICML.
- ▶ B. D. Ziebart, A. Maas, J. A. Bagnell, A. K. Dey (2008). *Maximum Entropy Inverse Reinforcement Learning*. AAAI. [CMU PDF](#)
- ▶ P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, D. Amodei (2017). *Deep Reinforcement Learning from Human Preferences*. NeurIPS. [arXiv:1706.03741](https://arxiv.org/abs/1706.03741)